Name: Harsh Patil
Class: D20C/25

Experiment No. 6

Aim:

Hands on Solidity Programming Assignments for creating Smart Contracts.

Theory:

1. Relevance of require Statements in Solidity Programs

The require statement acts as a **guard condition** within Solidity functions. It ensures that only valid inputs or authorized users can execute certain parts of the code.

- If the condition inside require evaluates to false, function execution **stops immediately**.
- All state changes made during that transaction are **reverted** to their original state.
- This rollback mechanism prevents invalid transactions from corrupting blockchain data.

Example uses in a Voting Smart Contract:

Check	**Example**
Caller has the right to vote	require(voters[msg.sender].weight > 0, "Has no right to vote");
Voter hasn't already voted	require(!voters[msg.sender].voted, "Already voted.");
Caller is the chairperson	require(msg.sender == chairperson, "Only chairperson can give right to vote.");

Summary: require statements enforce security, correctness, and reliability in smart contracts. They also let developers attach custom error messages, making debugging and contract interaction easier.

2. Key Solidity Concepts: mapping, storage, memory

a) mapping

A **mapping** links keys to values, similar to a hash table.

```
mapping(address => Voter) public voters;
```

- Each Ethereum address is mapped to a Voter struct.
- Useful in contracts like Ballot to associate voters with their data (whether they voted, their chosen proposal, etc.).
- Unlike arrays, mappings **have no length property** and **cannot be iterated directly** — gas-efficient for lookups but limited for enumeration.

b) storage

- Refers to the **permanent memory** of the contract, stored on the Ethereum blockchain.

- State variables declared at the contract level default to storage.
- Data is **persistent across transactions** — once written, it remains available unless explicitly changed.
- Writing to storage **consumes more gas** since it's permanent.
- *Example:* voter information saved in the voters mapping persists for the contract's entire lifecycle.

c) memory

- **Temporary storage**, valid only for the duration of a function call.
- Discarded once function execution ends.
- Used for temporary variables, function arguments, or intermediate computations.
- **Cheaper** than storage in gas cost.
- *Example:* proposal names or temporary string manipulations during a function call.

Summary: A developer must balance storage and memory usage to optimize efficiency and cost.

3. Why bytes32 Instead of string?

Earlier implementations of the Ballot contract used bytes32 for proposal names instead of string, primarily for **efficiency and gas optimization**.

bytes32

- Fixed-size type — always stores exactly 32 bytes.
- Simple storage, faster comparisons, lower gas costs.
- **Limitation:** proposal names capped at 32 characters — less flexible.

string

- Dynamically-sized type — can store text of variable length.
- Easier and more natural for developers/users to work with.
- **Limitation:** requires more complex handling inside the EVM, increasing gas usage and potentially slowing comparisons/manipulations.

Conversion

Modern implementations often convert bytes32 → string for better usability. Tools like the **Web3 Type Converter** help developers switch between these types during deployment and testing.

Code:

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity >=0.7.0 <0.9.0;

/**
 * @title Ballot
 * @dev Implements voting process
along with vote delegation
 */
contract Ballot {
    struct Voter {
        uint weight;        //
weight is accumulated by delegation
        bool voted;         // if
true, that person already voted
        address delegate;   //
person delegated to
        uint vote;          // index
of the voted proposal
    }

    struct Proposal {
        string name;        //
changed from bytes32 to string
        uint voteCount;     //
number of accumulated votes
    }

    address public chairperson;

    mapping(address => Voter) public
voters;

    Proposal[] public proposals;

    /**
     * @dev Create a new ballot to
choose one of 'proposalNames'.
     * @param proposalNames names of
proposals
     */
    constructor(string[] memory
proposalNames) {
        chairperson = msg.sender;
        voters[chairperson].weight =
1;
        for (uint i = 0; i <
proposalNames.length; i++) {

proposals.push(Proposal({
                name:
proposalNames[i],
                voteCount: 0
            }));
        }
    }

    function giveRightToVote(address
voter) external {
        require(
            msg.sender ==
chairperson,
            "Only chairperson can
give right to vote."
        );
        require(
            !voters[voter].voted,
            "The voter already
voted."
        );
        require(voters[voter].weight
== 0, "Voter already has the right
to vote.");
        voters[voter].weight = 1;
    }

    function delegate(address to)
external {
        Voter storage sender =
voters[msg.sender];
        require(sender.weight != 0,
"You have no right to vote");
        require(!sender.voted, "You
already voted.");
        require(to != msg.sender,
"Self-delegation is disallowed.");

        while (voters[to].delegate
!= address(0)) {
            to =
voters[to].delegate;
```

```
            require(to !=
msg.sender, "Found loop in
delegation.");
        }

        Voter storage delegate_ =
voters[to];
        require(delegate_.weight >=
1);

        sender.voted = true;
        sender.delegate = to;

        if (delegate_.voted) {

proposals[delegate_.vote].voteCount
+= sender.weight;
        } else {
            delegate_.weight +=
sender.weight;
        }
    }

    function vote(uint proposal)
external {
        Voter storage sender =
voters[msg.sender];
        require(sender.weight != 0,
"Has no right to vote");
        require(!sender.voted,
"Already voted.");
        sender.voted = true;
        sender.vote = proposal;

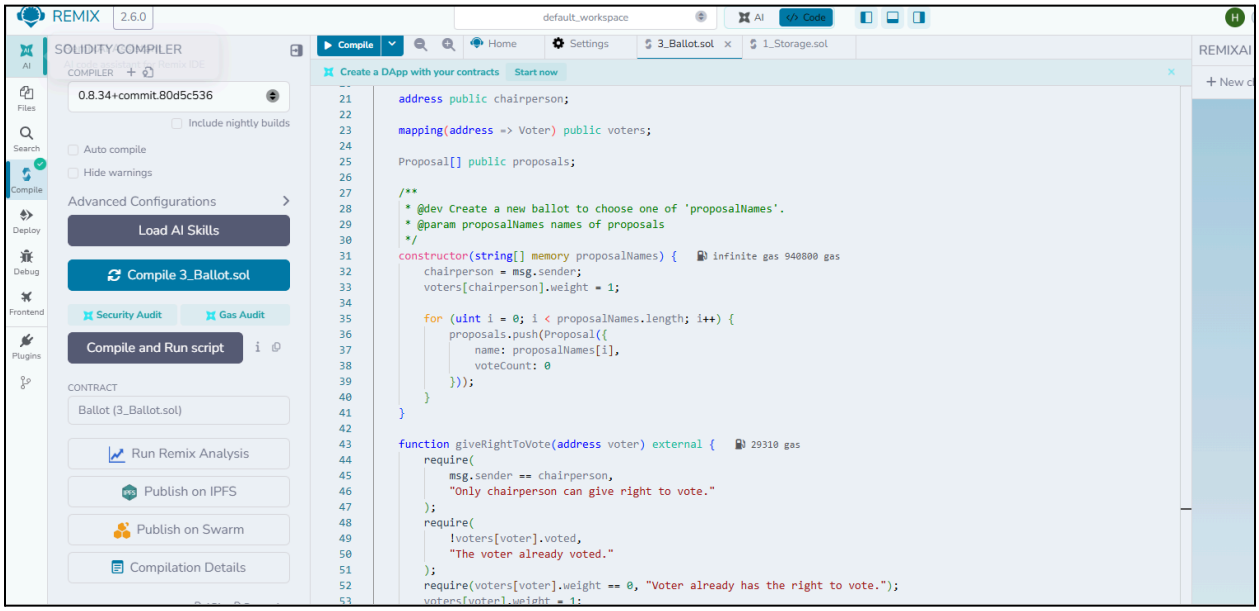

proposals[proposal].voteCount +=
sender.weight;
    }

    function winningProposal()
public view
        returns (uint
winningProposal_)
    {
        uint winningVoteCount = 0;
        for (uint p = 0; p <
proposals.length; p++) {
            if
(proposals[p].voteCount >
winningVoteCount) {
                winningVoteCount =
proposals[p].voteCount;
                winningProposal_ =
p;
            }
        }
    }

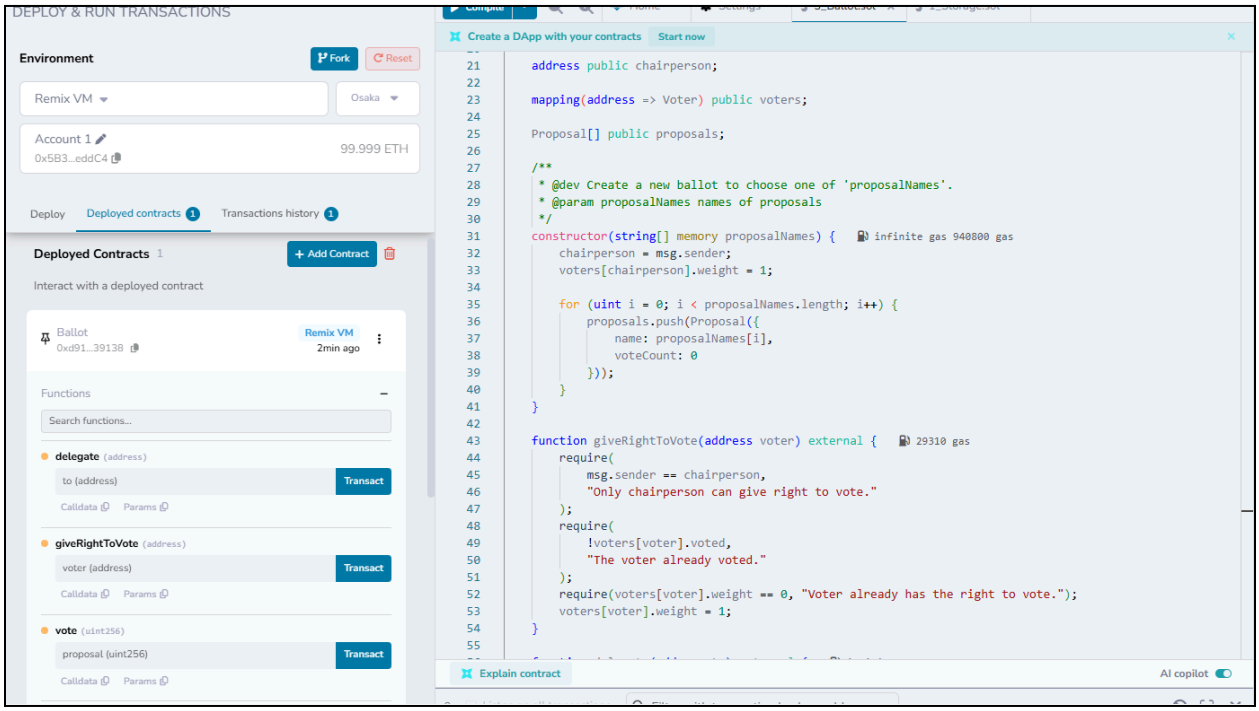
    /**
     * @dev Returns the name of the
winner
     * @return winnerName_ the name
of the winner
     */
    function winnerName() external
view
        returns (string memory
winnerName_)
    {
        winnerName_ =
proposals[winningProposal()].name;
    }
}
```

Output:

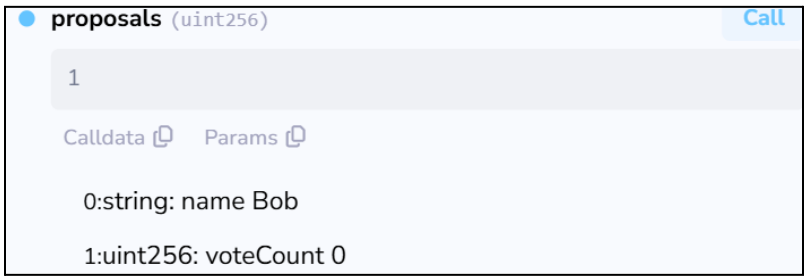
Contract Compiled:

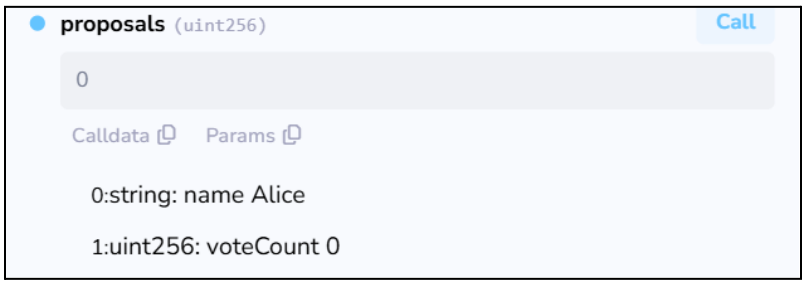


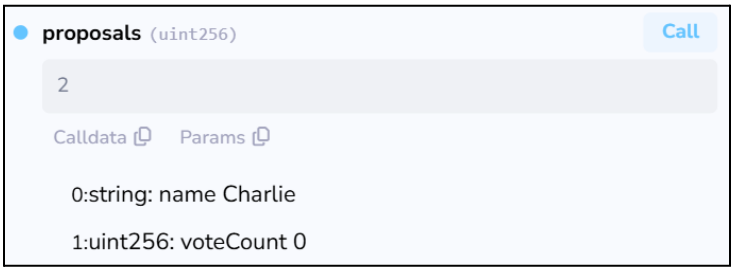
Contract Deployed:



Proposals:



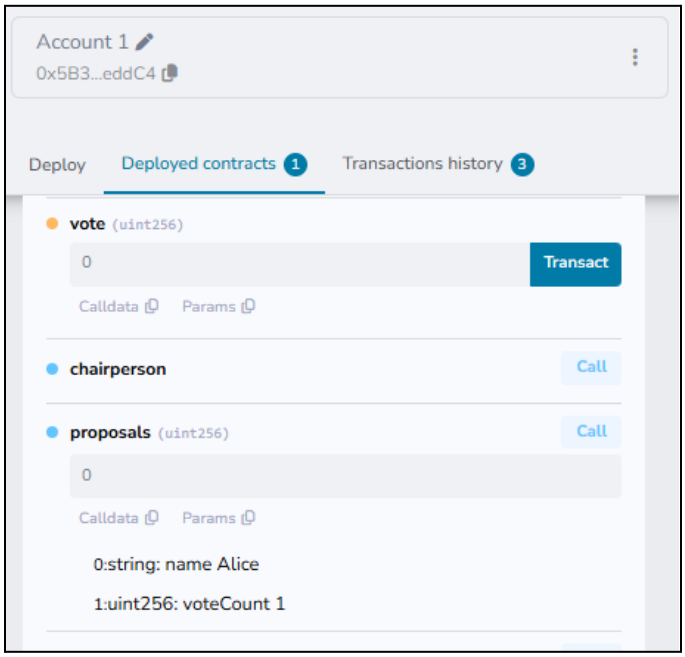




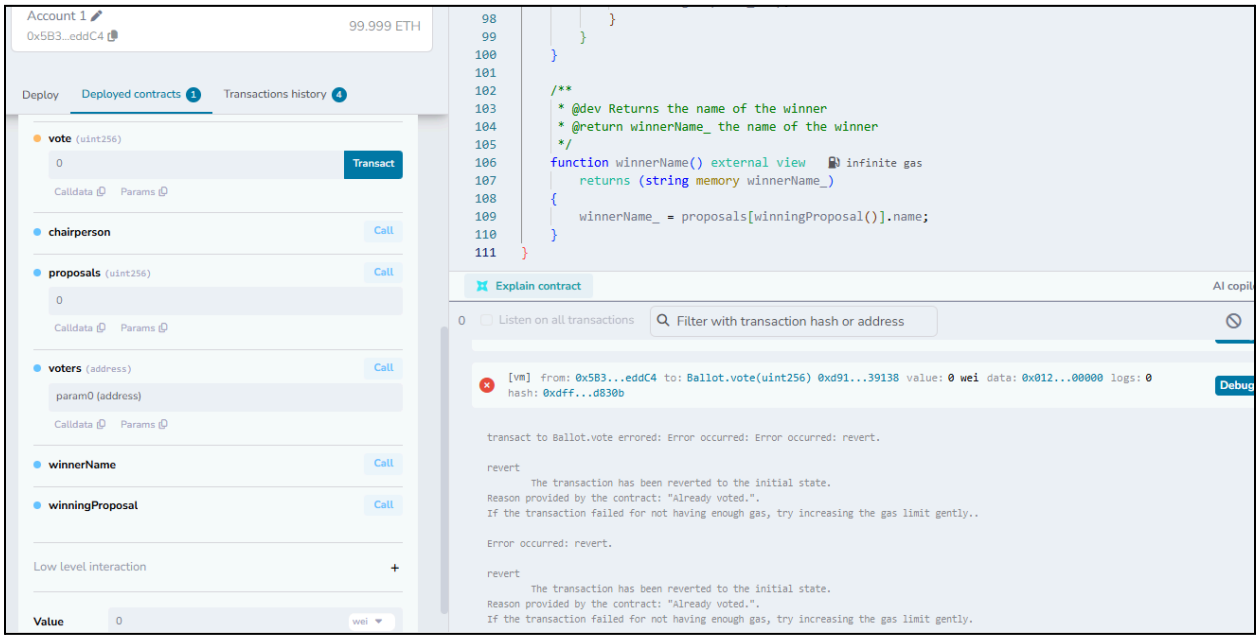
Chairperson/Contract Owner:



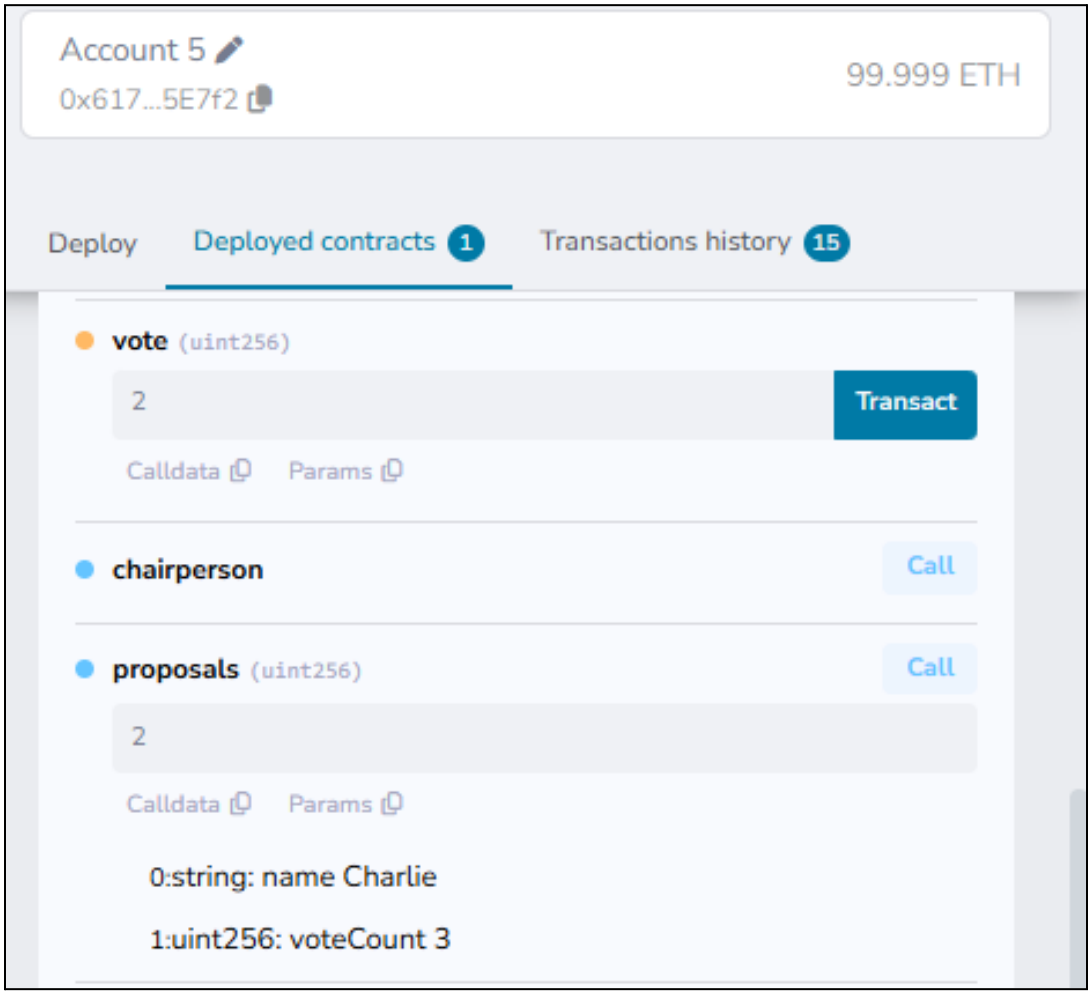
Vote for alice using account 1:



Prevent Double voting:

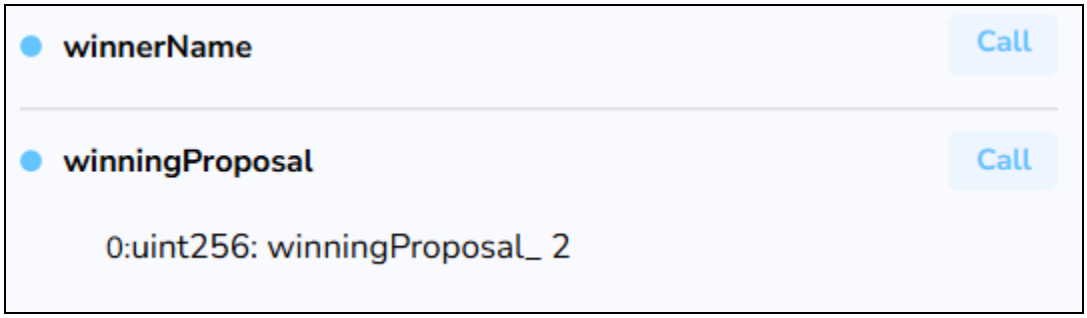


Give 3 votes to charlie using different Accounts:



Get Winner:


winnerName
Call
0:string: winnerName_ Charlie


winnerName
Call
winningProposal
Call
0:uint256: winningProposal_ 2